

Handling Data Inconsistencies Using Fuzzy Wuzzy

Introduction

In the world of data analytics, real-world datasets often contain **inconsistent or messy text data** due to typos, inconsistent formatting, or varying naming conventions (e.g., “McDonalds” vs. “McDonald's”). These inconsistencies can make tasks like merging datasets, grouping categories, or deduplication difficult.

Fuzzy matching helps solve this problem, and one popular tool for it is the **FuzzyWuzzy** library in Python, which uses **Levenshtein Distance** to calculate the similarity between strings.

Learning Objectives

By the end of this lesson, students will be able to:

1. Understand the concept of fuzzy string matching and its use in data cleaning.
 2. Use the FuzzyWuzzy library to identify similar strings.
 3. Apply fuzzy matching to clean and deduplicate inconsistent data.
 4. Integrate fuzzy matching into real-world preprocessing workflows.
-

Concept Explanation

Key Concepts

- **String Matching:** Comparing two strings to check if they are equal or similar.
- **Levenshtein Distance:** Number of edits (insertions, deletions, substitutions) to change one string into another.
- **Fuzzy Matching:** Scoring the similarity between two strings instead of checking for exact match.

- **FuzzyWuzzy Library:** A Python library built on top of `python-Levenshtein` that offers fuzzy string matching tools.

Real-World Applications

- Cleaning customer names in CRM systems
- Merging datasets with inconsistent city or product names
- Deduplicating product listings in e-commerce platforms
- Reconciling company names in financial datasets

Analogy

Think of fuzzy matching like **autocorrect** on your phone. Even if you type "Grammerly", it knows you meant "Grammarly" based on similarity — not exact spelling.

Python Code Examples

Basic Setup

```
python
CopyEdit
# Install the library
!pip install fuzzywuzzy[speedup]
```

Compare Two Strings

```
python
CopyEdit
from fuzzywuzzy import fuzz

str1 = "McDonalds"
str2 = "McDonald's"

similarity = fuzz.ratio(str1, str2)
print(f"Similarity score: {similarity}") # Output: Similarity score:
96
```

Partial Match

python

CopyEdit

```
fuzz.partial_ratio("Apple Inc.", "Apple Incorporated") # Output: 88
```

Token Sort Ratio

python

CopyEdit

```
fuzz.token_sort_ratio("fried chicken spicy", "spicy chicken fried") #  
Output: 100
```

Fuzzy Matching in a List

python

CopyEdit

```
from fuzzywuzzy import process  
  
choices = ["KFC", "Burger King", "McDonald's", "Mcdonalds", "Pizza  
Hut"]  
best_match = process.extractOne("McDonalds", choices)  
print(best_match) # ('McDonald's', 96)
```

Common Mistakes & Misunderstandings

Mistake	Clarification
Assuming fuzzy matching is 100% accurate	It's a <i>heuristic</i> , not a rule-based method. Always inspect results.
Using fuzzy matching when data can be standardized (like country codes)	Prefer exact matches when possible.
Matching across large datasets without optimizing	For large data, use <code>process.extractOne()</code> or vectorization techniques.

Interactive Exercise

Exercise 1: Clean Product Names

You have a product list with inconsistencies:

python

CopyEdit

```
raw_names = ["Nike Shoes", "nike shoe", "NIKE SHOE", "Nike's Shoe",  
"Niek Shoes"]
```

✅ Use FuzzyWuzzy to standardize all names to the most common one.

Exercise 2: Match to Master List

You have:

- Master list: ["Lays", "Doritos", "Cheetos", "Pringles"]
- Incoming data: ["lays chips", "dorittos", "chetoz", "pringls"]

Write code to map each to the closest correct brand name.

Student-Friendly Analogy

Imagine you're organizing a wedding guest list. Some people RSVP with "Mahnoor S.", some with "M. Salman", others as "Salman, Mahnoor". FuzzyWuzzy helps you group all of these as the same person — just like your mom would recognize all spellings of your cousin's name!

Recommended Resources

-  [FuzzyWuzzy GitHub](#)
 -  [Data School: Fuzzy String Matching](#)
 -  "Data Cleaning with Python and Pandas" by Datacamp or Towards Data Science blogs
 -  [RapidFuzz \(Faster Alternative\)](#)
-

Wrap-Up / Recap

- Fuzzy matching is essential for cleaning and unifying inconsistent text data.
 - FuzzyWuzzy uses Levenshtein distance to score string similarity.
 - Use different scoring methods like `ratio`, `partial_ratio`, `token_sort_ratio` for different contexts.
 - Always validate your matches with manual checks or thresholds.
-

Quiz or Reflection Questions

1. What is Levenshtein Distance and why is it used in FuzzyWuzzy?
2. When should you prefer fuzzy matching over exact matching?
3. What's the difference between `ratio` and `token_sort_ratio`?
4. How would you handle a large dataset where fuzzy matching is slow?